CSCI-102 Data Structures - Fall 26 - Quiz 2 NetID: _____

Full Name: _____

The quiz contains 10 questions. Question 10 is worth 2 points, all others 1 point. The total is 11 points and the score is out of 10 (so you still get max points if you lose one point).

One handwritten A6 (or 4x6 index card) cheat sheet allowed, no printed material. Remember that your electronic devices should be silent and put away in your bag at all times.

---

1. Two algorithms A and B perform the same task. Algorithm A has a worst-case complexity of $O(N^3)$. Algorithm B has a worst-case complexity of $O(2^N)$. Assume that both algorithms are executed with the same inputs on the same computer and their exact running times are measured.
Which of the following are true (select one or more):

- ☒ there is a value N_0 such that algorithm A is faster than algorithm B for every $N > N_0$
- ☐ there is a value N_0 such that algorithm B is faster than algorithm A for every $N > N_0$
- ☒ it is possible that algorithm A is always faster than algorithm B for this specific problem
- ☐ algorithm B is faster than algorithm A for $N = 2$
- ☐ there must exist some N for which A and B are equally as fast

---

2. Bubble sort is used to sort the array `{5, 1, 4, 2, 8, 3}` in ascending order. Each pass goes through the array from left to right and swaps adjacent elements that are out of order.
What does the array look like after the first two full passes?
Answer: _____ ({1, 2, 4, 3, 5, 8})

---

3. A sorting algorithm is **adaptive** if (select one):

- ☐ equal elements keep their original relative order
- ☒ it runs faster on input that is already partly sorted
- ☐ it uses only O(1) extra memory
- ☐ it can sort any type that implements `Comparable`
- ☐ it switches to a different algorithm for small subarrays

---

4. Consider the following classes.

```
class Student {
    String name;
    double gpa;
    Student(String name, double gpa) { this.name = name; this.gpa = gpa; }
}

class ByGpa implements Comparator<Student> {
    public int compare(Student a, Student b) {
        return Double.compare(b.gpa, a.gpa);
    }
}
```

The array below is sorted with a **stable** sorting algorithm, using a `ByGpa` object to compare elements.

```
Student[] roster = { new Student("Ann", 3.2), new Student("Bo", 3.9),
                     new Student("Cy", 2.8), new Student("Dee", 3.2) };
```

Hint: note the order of `b.gpa` and `a.gpa` in `compare`, and remember that a sorting algorithm sorts things in ascending order.
In which order are the names after sorting?
Answer: _____ (Bo, Ann, Dee, Cy)

---

5. The following method should sort an array in ascending order using insertion sort, but one line contains a bug. Which change fixes the method (select one)?
Hint: you can simulate by hand what the method would do on a very small array to pin down which operation is buggy.

```
1  void insertionSort(int[] a) {
2      for (int i = 1; i < a.length; i++) {
3          int current = a[i];
4          int j = i - 1;
5          while (j >= 0 && a[j] > current) {
6              a[j] = a[j + 1];
7              j--;
8          }
9          a[j + 1] = current;
10     }
11 }
```

- ☐ line 2 becomes `for (int i = 0; i < a.length; i++){`
- ☐ line 3 becomes `int current = a[i - 1];`
- ☐ line 5 becomes `while (j >= 0 && a[j] < current){`
- ☒ line 6 becomes `a[j + 1] = a[j];`
- ☐ line 9 becomes `a[j] = current;`

---

6. What does the following code fragment print?
Hint: this code compiles and runs without errors, and the last line will just print the list elements in square brackets and separated by commas.
Hint: write down the state of the list after each call on scrap paper.

```
List<String> list = new ArrayList<>();
list.add("a");
list.add("b");
list.add("c");
list.add("d");
list.add(1, "e");
list.remove(3);
list.add(list.size(), "f");
list.set(0, list.get(2));
System.out.println(list);
```

Answer: _____ ([b, e, b, d, f])

---

7. The following method of an array-based list of `double`s removes and returns the element at position `pos`, moving the elements after it down by one position. `data` is the array holding the elements and `size` is the number of elements in the list. Complete the missing line, so that the method works correctly.

```
double remove(int pos) {
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();
    double removed = data[pos];
    for (int i = pos; i < size - 1; i++)
        ---
    size--;
    return removed;
}
```

Answer:

☒ `data[i] = data[i + 1];`
☐ `data[i + 1] = data[i];`
☐ `data[i] = data[i - 1];`
☐ `data[pos] = data[i + 1];`
☐ `data[i] = data[size - 1];`

---

8. Appending an element to a `java.util.ArrayList` with `add(e)` takes O(1) **amortized** time. This means that (select one):

☐ every call to `add(e)` takes O(1) time in the worst case
☐ a call to `add(e)` takes O(1) time on average, over random inputs
☒ any sequence of M calls to `add(e)`, starting from an empty list, takes O(M) time in total
☐ a call to `add(e)` takes O(1) time as long as the list has fewer than some fixed number of elements
☐ the cost of a single call to `add(e)` does not depend on the capacity of the array

---

9. `arraylist` is a `java.util.ArrayList` containing N elements, and the following method is called on it.

```
void process(ArrayList<Integer> arraylist) {
    arraylist.add(0, -1);
    for (int i = 0; i < arraylist.size(); i++)
        arraylist.set(i, arraylist.get(i) * 2);
    while (arraylist.size() > 0)
        arraylist.remove(arraylist.size() - 1);
}
```

What is the worst-case complexity of `process`, as a function of N?
Answer: _____ (O(N): `add(0, -1)` is O(N) once, the loops do N+1 iterations of O(1) operations each, since `get`, `set` and removing the last element are O(1))

---

10. (2 points) Draw how merge sort sorts the array `{8, 3, 5, 1, 7, 2, 6, 4}`: every split, down to subarrays of one element, and every merge, up to the sorted array. Write the content of each subarray.

```
          [8 3 5 1 7 2 6 4]
      [8 3 5 1]       [7 2 6 4]
    [8 3]   [5 1]   [7 2]   [6 4]
  [8] [3] [5] [1] [7] [2] [6] [4]
    [3 8]   [1 5]   [2 7]   [4 6]
      [1 3 5 8]       [2 4 6 7]
          [1 2 3 4 5 6 7 8]
```
